CSE 213 — Software Engineering

Software Ethics and Software Requirement Analysis

Lesson notes · Exam approach · TF practice with solutions · Quick notes (IJ Mam's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — ethics and requirement questions
Lesson 21 — Software Ethics: Dimensions, Cases and the IEEE-CS/ACM Code — 8 - Software Ethics pp. 1–12
Lesson 22 — Requirement Analysis: Elicitation, Specification, Quality, MoSCoW, RTM — Software Requirement Analysis pp. 1–12
Quick Notes — Ethics and Requirements — revision

NOTE Slide numbers refer to the PDF page numbers of the deck named under each lesson title. TF page numbers refer to 'CSE 213 All Past Papers.pdf' (70 pages); the newest paper (22 batch, labelled 2024-25 here) is a separate file, cited as '22 batch p.N'. Old course code: CSE 307. Lesson 21: '8 - Software Ethics.pdf' (12 pages). Lesson 22: 'Software Requirement Analysis.pdf' (12 pages). JRP and the classic fact-finding list are textbook background (Whitten & Bentley) for the older papers.

Exam Map — What the TF Papers Ask

Session (paper)	Questions in this note file
2024-25 (CSE 213)	Q1(b) Security engineer leaks unpatched vulnerabilities to the media — which ethical principles are breached? (10) [L21] · Q5(c) Startup with no SRS: how to prepare the requirement specification (10) [L22]
2023-24 (CSE 213)	Q6(a) Health app: ethical breaches (8) [L21] · Q6(b) 'Act consistently with the public interest' — explain (7) [L21]
2021-22 (CSE 307)	Q2(c) Functional vs non-functional requirements (5) [L22] · Q4(b) Assess the requirement 'update notification after a fixed time…' against quality measures (10) [L22]
2014-15 (CSE 307)	Q6(c) Training school: fact-finding approach (15) [L22]
2013-14 (CSE 307)	Q1(a) Government filing system: roles of fact finding, questionnaires, interviews (12) [L22] · Q1(b) Design a JRP session: roles and responsibilities (12) [L22] · Q5(d) Fact-finding methods (14) [L22] · Q5(e) JRP: advantages and disadvantages (10⅔) [L22]
2012-13 (CSE 307)	Q5(d) JRP and its participants (4⅔) [L22] · Q7(b) Health centre: seven fact-finding methods per user (15) [L22]
2011-12 (CSE 307)	Q5(a) Classify ten requirement statements into seven categories (10) [L22]
2010-11 (CSE 307)	Q7(a) Professional and ethical responsibilities of a software engineer (6⅔) [L21] · Q7(b) Land-management computerisation: fact-finding techniques (25) [L22]

EXAM FOCUS Ethics is new and ★★★: scenario questions in 2024-25 (leaked vulnerabilities) and 2023-24 (health app, 'public interest'). Requirements: 2024-25 (no SRS — how to specify), 2021-22 (quality of a requirement statement, functional vs non-functional), and fact-finding in many older papers. Learn the 8 principles with one-line meanings and the six qualities of a good requirement; answer scenarios principle by principle.

Lesson 21 — Software Ethics: Dimensions, Cases and the IEEE-CS/ACM Code

Pages 1–12 (8 - Software Ethics)

Building software is never value-neutral (p. 2). The fundamental question (p. 3): **'Does this system do what it should, for whom it should, without harm?'** Every architectural and algorithmic choice encodes human values. Ethical codes need moral judgement of trade-offs, not mechanical rules; engineers must put public safety and rights above deadline pressure; ignoring known risks is negligence.

Dimension (pp. 4–8)	Principle	Case study
1 Data privacy	informed, explicit consent; collect only necessary data	Cambridge Analytica 2018: 87 M Facebook users' data harvested via quiz apps; $5 B FTC fine
2 Bias & fairness	audit systems against systemic discrimination	Amazon AI recruiting 2018 penalised résumés containing 'women's'; scrapped
3 Accountability	'the algorithm did it' is no defence; humans own outcomes	VW Dieselgate (defeat software, 11 M cars); Boeing 737 MAX MCAS (346 deaths)
4 Transparency	auditable, explainable decisions users can contest	Dutch childcare benefits scandal 2021: opaque risk algorithm, government resigned
5 Autonomy	no deceptive dark patterns	FTC v. Amazon 2023 'Iliad' Prime cancellation flow
6 Security duty	negligent flaws are ethical failures	Therac-25 race conditions (fatal overdoses); Equifax 147 M records, unpatched Apache Struts
7 Sustainability	minimise energy/carbon of computation	Strubell 2019: training one large model ≈ CO_2 of 5 cars' lifetimes
8 IP & authorship	fair licensing in generative AI	NYT v. OpenAI; GitHub Copilot lawsuits

IEEE-CS / ACM Software Engineering Code — 8 principles (p. 9)

#	Principle	Meaning
1	PUBLIC (top priority)	act consistently with the public interest — public health, safety and welfare
2	CLIENT & EMPLOYER	serve client and employer loyally, consistent with the public interest
3	PRODUCT	products meet the highest professional standards; rigorous QA
4	JUDGMENT	maintain integrity and independence of professional judgement
5	MANAGEMENT	managers promote an ethical approach to development and maintenance
6	PROFESSION	advance the integrity and reputation of the profession
7	COLLEAGUES	be fair to and supportive of colleagues
8	SELF	lifelong learning; promote an ethical approach to practice

Practical contrast (p. 10): dark patterns vs transparent reversible choices; uninformed harvesting vs privacy-by-design; unverified biased training data vs fairness auditing; shipping known vulnerabilities vs safety gates and continuous patching. Ethics is a design constraint, not an afterthought — ask 'who could this harm, and how would we know?' (p. 12).

Exam Approach

1. List each action in the scenario.
2. For each, name the principle(s) of the Code and the dimension(s) involved, and say breached or not, with why.
3. Weigh the competing duties (public vs employer, confidentiality vs disclosure).
4. Say what the ethical course of action would have been.

Practice — Lesson 21

(a) Actual — 2024-25 Q1(b). (b) Actual — 2023-24 Q6(a). (c) Actual — 2023-24 Q6(b). (d) Actual — 2010-11 Q7(a).

(a) A security engineer hired after the first release of a government citizen-information platform finds critical flaws that could expose sensitive data, and immediately discloses them anonymously to the media, claiming a moral duty to protect citizens; the scandal damages the firm and disrupts the project. Any ethical breaches? Justify. (10)

(b) A start-up health app collects heart rate, activity and diet; engineers built the suggestion algorithm by studying articles; it warns of heart risks and recommends medicines; it shares anonymised health data with consent, but also age, profession and address with healthcare institutions. Identify ethical breaches and why they are unethical. (8)

(c) Explain 'act consistently with the public interest'. (7)

(d) Professional and ethical responsibilities of a software engineer. (6⅔)

✓ Solution

(a) Yes. The motive (protect the public, Principle 1) is legitimate, but the method breaches: **Client & Employer (2)** — confidential information of the employer and client disclosed without authorisation; **Judgment (4)** and **Profession (6)** — no responsible disclosure, public release of unpatched vulnerabilities **increases** risk because attackers learn them before fixes exist (so it also works against Principle 1); **Colleagues (7)** — the team got no chance to fix. Ethical path: report immediately and in writing to the employer/client with severity and a deadline; push for a fix and user notification; if management ignores a serious public risk, escalate to the government authority or a CERT (coordinated disclosure), going public only as a last resort and without technical details that enable exploitation.

(b) (1) **Product/Public** — medical warnings and medicine recommendations from an algorithm built by non-clinicians from articles, apparently not validated by doctors or regulators: unsafe advice can harm users. (2) **Privacy (dimension 1)** — age, profession and address shared with institutions: these are identifying data, so the 'anonymised' health data can be re-identified; consent covered only anonymised health data, so this exceeds consent and violates data minimisation. (3) **Transparency/Accountability** — users are not told how suggestions are made or who is responsible. (4) **Judgment/Profession** — engineers acting outside their competence. Consent for anonymised data sharing is the one ethical element.

(c) Principle 1 of the Code: the engineer's first duty is to the public — health, safety and welfare. Accept full responsibility for one's work; approve software only if it is safe, meets specs, passes tests and does not diminish quality of life, privacy or the environment; disclose to appropriate persons any actual or potential danger; be fair and avoid deception in public statements; consider the disadvantaged. When client/employer interests conflict with public safety, the public interest wins (e.g. refusing to ship known safety defects).

(d) The eight principles with one line each, plus professional responsibilities: confidentiality, competence (do not accept work beyond competence), intellectual-property rights, not misusing computers, honest estimates and reports.

Lesson 22 — Requirement Analysis: Elicitation, Specification, Quality, MoSCoW, RTM

Pages 1–12 (Software Requirement Analysis)

Poor requirement analysis is a major cause of project failure (p. 2): misaligned features (rewrites), architectural churn (mid-way changes break structure), resource drain (missed deadlines, budget overruns, burnout). Benefits: fixing errors early is far cheaper than post-release hot-fixes; one source of truth aligns stakeholders.

```
Elicitation --> Analysis & refinement --> Specification --> Validation          (p. 3)
gather raw needs   resolve conflicts,          precise formal      verify spec against
& constraints      model behaviour             criteria (SRS)       stakeholder objectives
```

Elicitation technique (pp. 4–6)	Best for	Watch out for
1-on-1 interviews	deep needs, domain rules	interviewer bias; does not scale
Observation / field study	real workflows and workarounds	time-consuming; observer effect
Surveys / questionnaires	many users, quantifiable views	shallow answers; low response
Workshops (co-design, focus groups)	conflicting stakeholders, quick consensus	dominant voices; scheduling

Functional (what the system must do)	Non-functional (how well it performs)
send money, check balance, cash-in via agent, view transaction history	works offline, loads in 3 s on 3G, usable by low-literacy users, audit logging (performance, usability, security, reliability, availability, portability, maintainability)

Representation (p. 8): use case diagrams, BPMN & DFDs, UML class diagrams. **MoSCoW** prioritisation (p. 9): Must have (non-negotiable for release), Should have (high value, workaround possible), Could have (if resources permit), Won't have (out of this cycle).

MEMORIZE A good requirement is **unambiguous, verifiable, complete, consistent, measurable, feasible** (p. 10). Weak: 'The app should be fast and easy to use for rural users.' Testable: 'A balance check completes in ≤ 3 s on a 3G link for 95 % of requests; 8 of 10 first-time users send money unaided in a pilot test.'

Requirements Traceability Matrix (p. 11): bi-directional tracing from business goals forward to test cases and back to origin; impact analysis before approving changes; verification coverage (no orphan features, no missing tests).

NOTE Textbook background for older papers. Fact-finding methods (seven): sampling of existing documents/forms/databases, research and site visits, observation of the work environment, questionnaires, interviews, prototyping (discovery prototyping = building a quick prototype to discover requirements), joint requirements planning (JRP). **JRP**: a structured multi-day workshop — sponsor, facilitator, users and managers, scribes, IT staff — that defines requirements together. + faster, shared ownership, fewer conflicts; − expensive, needs a skilled facilitator, people away from work, dominant voices.

Practice — Lesson 22

(a) Actual — 2024-25 Q5(c). (b) Actual — 2021-22 Q4(b). (c) Actual — 2021-22 Q2(c). (d) Actual — 2011-12 Q5(a). (e) Actual — 2013-14 Q1(a)/(b), Q5(d)–(e); 2012-13 Q5(d), Q7(b); 2010-11 Q7(b); 2014-15 Q6(c).

(a) A start-up built a product without any written specification; prepare a proper specification so the intended behaviour is documented and agreed for future review. How would you proceed? (10)

(b) Assess 'The software should provide update notification after a fixed time. Users should be able to perform action based on the notification' against requirement quality measures. (10)

(c) Differences between functional and non-functional requirements. (5)

(d) Classify: (i) authenticate user each session (ii) ≤ 2 restart failures per month (iii) ≤ 5 % OS-specific code (iv) cyclomatic complexity ≤ 6 (v) run within 250 MB RAM and 2 GB disk (vi) rocket reaches 11.2 km/s (vii) runs on XP, Vista, 7 (viii) < 5 clicks to add a weekly meeting (ix) after 2 h training add a customer in 3 min (x) 100 ticket requests per minute at peak — into performance 1, portability 2, security 3, usability 4, availability 5, maintainability 6, domain 7. (10)

(e) Fact-finding: describe the methods / choose methods for each user / roles of interviews, questionnaires / JRP session, participants, advantages and disadvantages. (5–25)

✓ Solution

(a) Reverse-engineer and then validate: (1) identify stakeholders (founders, users, support, developers); (2) elicit — interview founders and developers, observe users with the live product, survey users, read the code, database schema, tickets, release notes and help pages; (3) analyse — model the current behaviour with use cases, BPMN/DFD and a class/domain model, list functional and non-functional requirements, resolve conflicts between 'what it does' and 'what it should do'; (4) specify — write an SRS (IEEE 830 style: purpose, scope, definitions, overall description, numbered functional requirements, NFRs with measurable criteria, constraints), prioritise with MoSCoW; (5) validate — review with stakeholders, check each requirement is unambiguous, verifiable, complete, consistent, measurable, feasible, and confirm against the running system with tests; (6) baseline it, create an RTM linking requirements to code modules and test cases, and put the SRS under version control for future changes.

(b) Ambiguous: 'update notification' (about what? app update or data update?), 'fixed time' (how long, from when, configurable?), 'perform action' (which actions?). Not verifiable/measurable: no time value or success criterion. Incomplete: channel (push/e-mail), recipients, behaviour if offline or dismissed. Consistency and feasibility cannot be judged. Rewrite: 'When a new app version is published, the system shall send a push notification to every active user within 24 hours; the notification shall offer Update now and Remind me later; choosing Update now starts the update within 5 s.'

(c) Functional = services/behaviour the system must provide (inputs, outputs, functions) — 'the system shall transfer money between accounts'. Non-functional = qualities and constraints on how it provides them (performance, security, usability, reliability, portability) — 'a transfer shall complete within 3 s for 95 % of requests'. Functional requirements are tested by functional tests; NFRs need measurable criteria and often affect the whole architecture.

Statement	i	ii	iii	iv	v	vi	vii	viii	ix	x
Category	3 security	5 availability	2 portability	6 maintainability	1 performance	7 domain	2 portability	4 usability	4 usability	1 performance

(e) Use the NOTE: describe the seven methods; map them to users — doctors and pharmacists: short interviews and observation; nurses, receptionists, attendants: observation + questionnaires; patients: questionnaires/surveys; managers: interviews and JRP; existing forms: sampling of documents. For a government filing system: document sampling and observation to learn current files, questionnaires to many clerks for volumes and problems, interviews with section heads for rules and exceptions. JRP session: sponsor opens, facilitator runs, users/managers state needs, scribe records, IT staff advise feasibility; room with whiteboards, agenda, 2–4 days.

Quick Notes — Ethics and Requirements

Revision points for Lessons 21–22.

L21 · Ethics

- Question: does it do what it should, for whom, without harm?
- 8 dimensions: privacy, bias, accountability, transparency, autonomy, security, sustainability, IP.
- Code: Public, Client & Employer, Product, Judgment, Management, Profession, Colleagues, Self.
- Disclosure dilemma: report internally → coordinated disclosure → public only as last resort.

L22 · Requirements

- Elicitation → analysis → specification → validation.
- Interviews, observation, surveys, workshops (+ document sampling, prototyping, JRP).
- Good requirement: unambiguous, verifiable, complete, consistent, measurable, feasible.
- MoSCoW; RTM traces goals ↔ requirements ↔ tests.